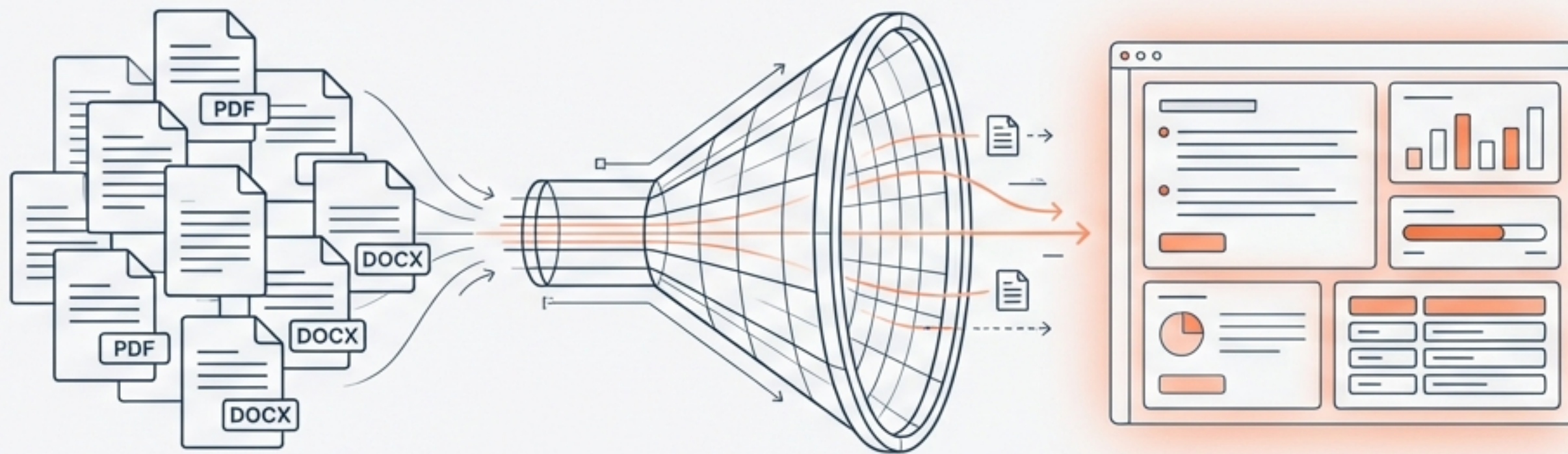
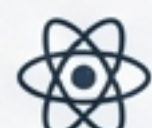




Helvetica Now Display

생성형 AI 기반 문서 요약 시스템 개발

React, FastAPI, LLM(ChatGPT/Ollama)을 활용한 풀스택 구현 가이드



React



FastAPI

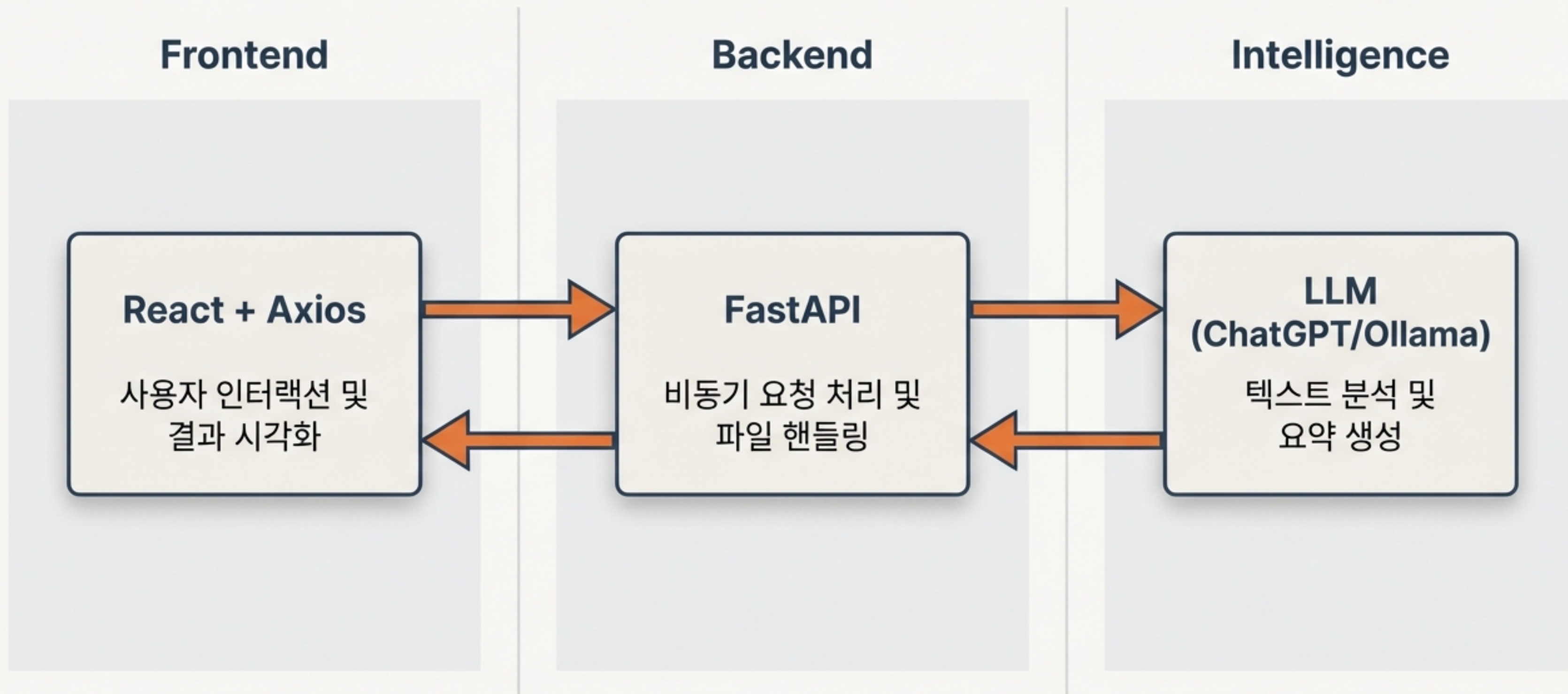


OpenAI

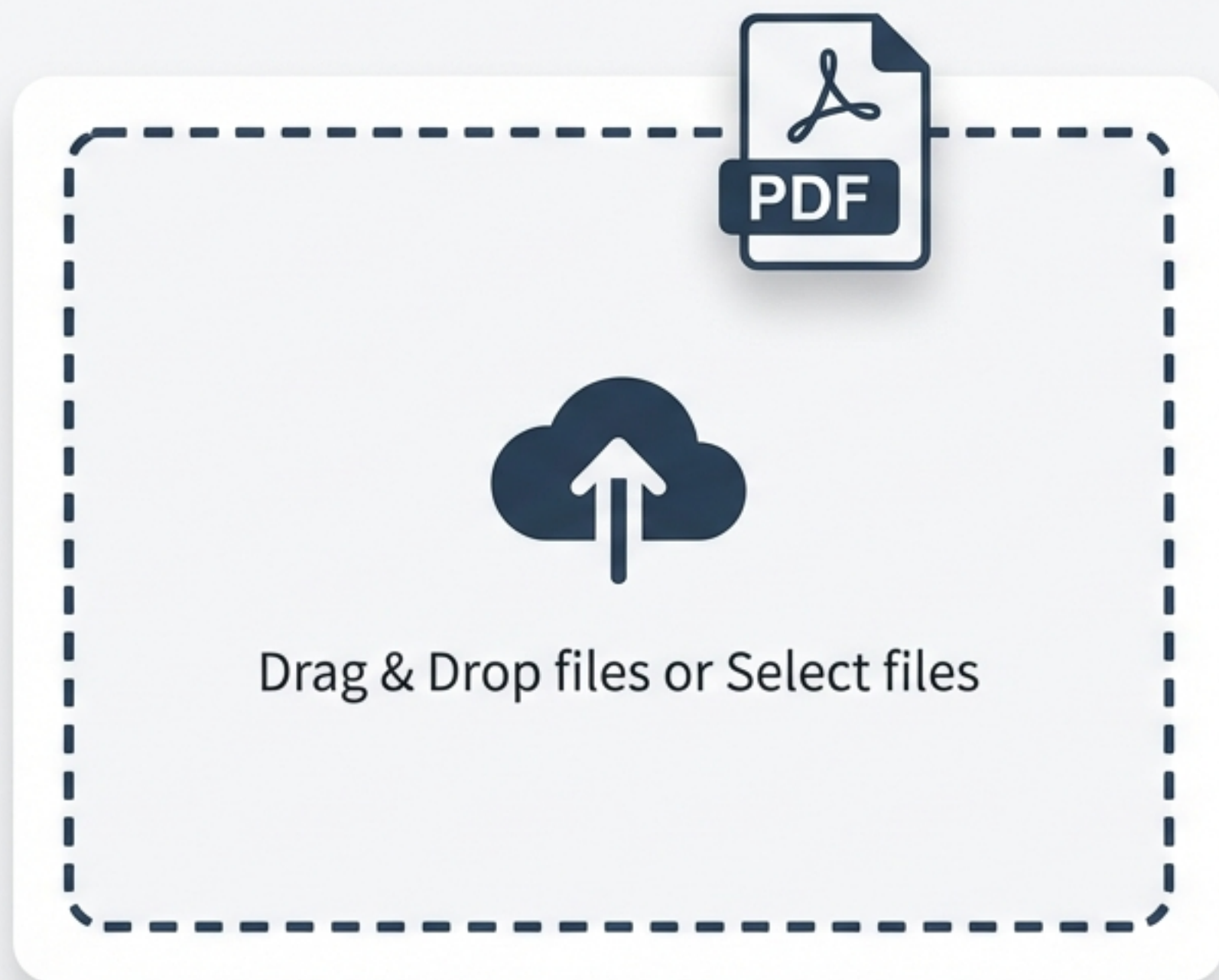


Ollama

시스템 아키텍처 및 데이터 파이프라인



[Frontend] 파일 업로드 컴포넌트 구현



```
FileUpload.jsx

const onDrop = useCallback(acceptedFiles => {
  // 파일 유효성 검사 및 State 업데이트
  setFiles(prev => [...prev, ...acceptedFiles]);
}, []);

// Dropzone component usage
<div {...getRootProps()} className="dropzone">
  <input {...getInputProps()} />
</div>
```

- 드래그 앤 드롭(Drag & Drop) 이벤트 처리
- 파일 확장자 제한 (.pdf, .docx, .txt)
- `useCallback` 훅을 통한 성능 최적화

[Frontend] Axios Interceptor를 활용한 UX 최적화

개별 API 요청마다 로딩 상태를 관리하는 대신, 인터셉터를 통해 전역 로딩(Global Spinner) 처리.

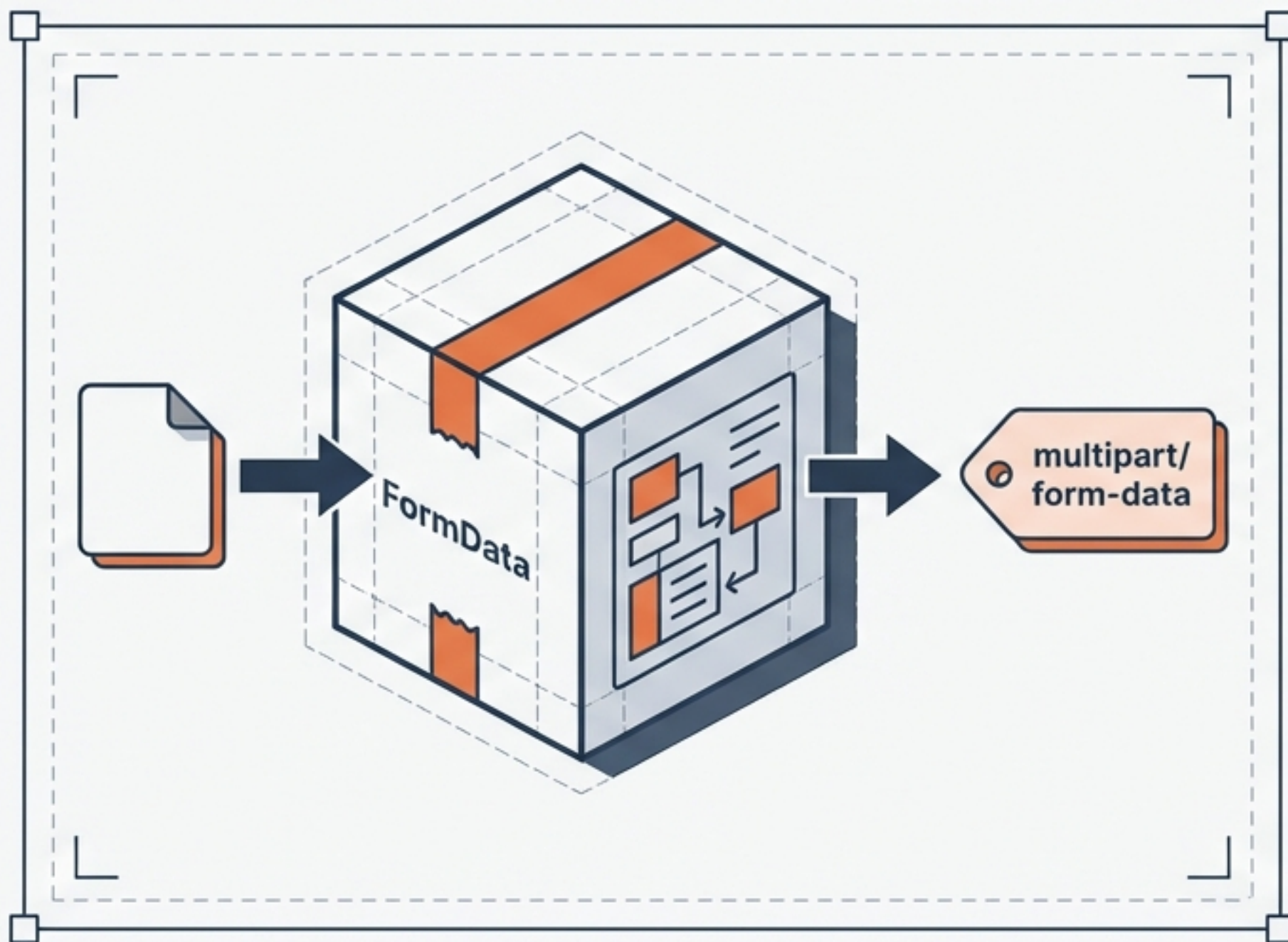


```
// Request Interceptor: 로딩 시작
axios.interceptors.request.use(config => {
  setLoading(true);
  return config;
});

// Response Interceptor: 로딩 종료
axios.interceptors.response.use(res => {
  setLoading(false);
  return res;
});
```


[Integration] 서버로 파일 전송

서버로 파일 데이터를 효율적으로 전송하기 위해 FormData와 multipart/form-data를 활용합니다.



FileUpload.js

```
const handleUpload = async () => {  
  const formData = new FormData();  
  formData.append("file", file); // 'file' matches backend param  
  
  const res = await axios.post("/summarize", formData, {  
    headers: {  
      "Content-Type": "multipart/form-data"  
    }  
  });  
};
```

- 브라우저의 'FormData' 객체 생성
- 'multipart/form-data' 헤더 설정 (필수)
- '/summarize' 엔드포인트 호출

[Backend] FastAPI 엔드포인트 설계

Shift focus to the server side (Python environment).



UploadFile: 파일을
메모리에 전부 로드하지
않고 Spooled 방식 처리
(대용량 파일 대응).



async def: 비동기
처리로 I/O 블로킹 방지.

fastapi_endpoint.py

```
from fastapi import FastAPI, UploadFile, File

@app.post("/summarize")
async def summarize_document(file: UploadFile = File(...)):
    content = await file.read()
    # 텍스트 추출 및 AI 처리 로직 연결
    return {"filename": file.filename, "summary": result}
```


[Backend] 문서 텍스트 추출 및 전처리



UI card

```
if file.filename.endswith(".pdf"):
    reader = PdfReader(io.BytesIO(content))
    text = "".join([page.extract_text() for page in reader.pages])
```


[AI] 프롬프트 엔지니어링 전략

Designing prompt structures for specific AI responses.

Prompt Card

You are a document analyst. Analyze the text and return a JSON object with:

1. 'summary': A 3-sentence summary.
2. 'keywords': A list of 5 key terms with 'text' and 'value' (score).

Output must be valid JSON.



Target Output

```
{  
  "summary": "...",  
  "keywords": [  
    { "text": "...", "value": 0.95 },  
    { "text": "...", "value": 0.87 },  
    // ...  
  ]  
}
```

목표: 단순 텍스트 요약이 아닌, UI 시각화를 위한 구조화된 데이터(JSON) 확보.

[Backend] LLM API 연동

Python Code

```
import openai

client = openai.OpenAI(
    # 환경 변수에 따라 OpenAI 또는 Local Ollama URL 사용
    base_url="http://localhost:11434/v1",
    api_key="ollama"
)

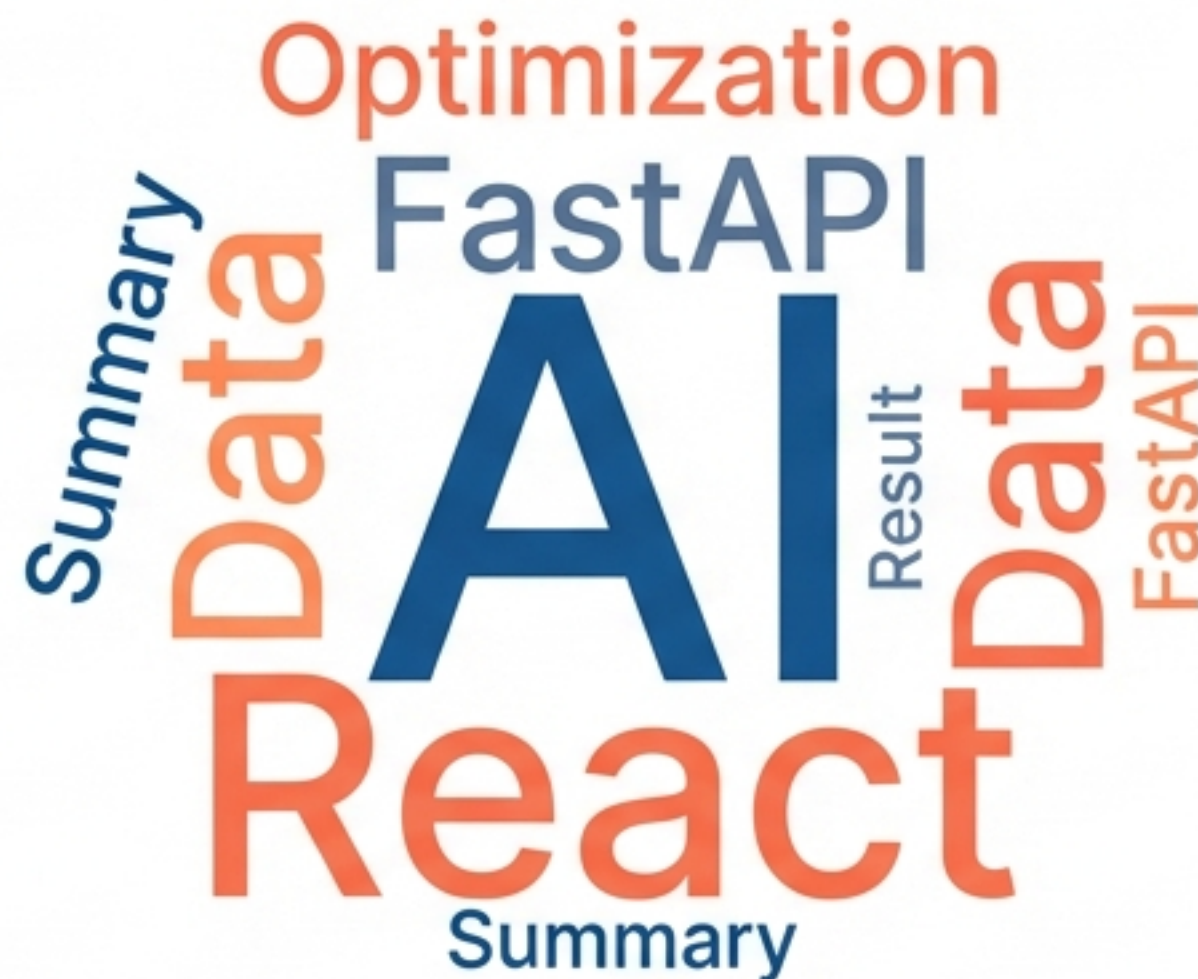
response = client.chat.completions.create(
    model="llama3", # or "gpt-4o"
    messages=[{"role": "user", "content": prompt}],
    response_format={"type": "json_object"}
)
```

환경 변수에 따라 OpenAI 또는 로컬 Ollama 모델 선택 가능.

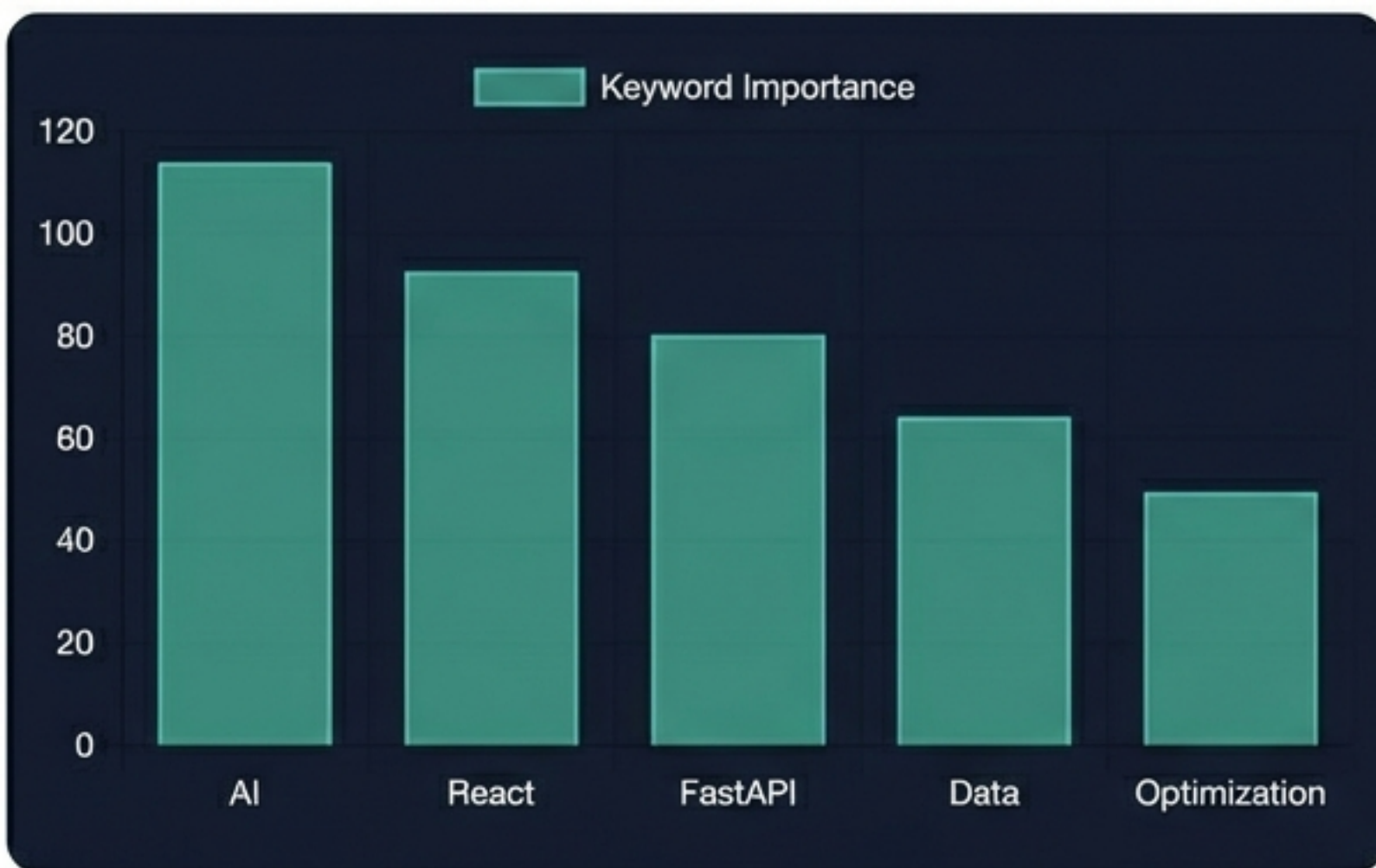
[Frontend] 결과 시각화 - 워드 클라우드

```
import ReactWordcloud from 'react-wordcloud';

// Data format: { text: "AI", value: 90 }
<div style={{ height: 400, width: 600 }}>
  <ReactWordcloud words={keywords} />
</div>
```



[Frontend] 데이터 시각화 - 차트 구현



```
import { Bar } from 'react-chartjs-2';

const data = {
  labels: keywords.map(k => k.text),
  datasets: [{
    label: 'Keyword Importance',
    data: keywords.map(k => k.value),
    backgroundColor: 'rgba(75, 192, 192, 0.6)'
  }]
};
```

키워드 중요도(Score) 또는 주제별 분포를 막대/파이 차트로 표현.

[Frontend] 요약 결과 다운로드 기능



```
const downloadTxt = () => {  
  const blob = new Blob([summaryText], { type: "text/plain" });  
  const link = document.createElement("a");  
  link.href = URL.createObjectURL(blob);  
  link.download = `${originalFileName}_summary.txt`;  
  link.click();  
};
```

Blob 객체와 URL.createObjectURL을 사용하여 브라우저 메모리 내에서 파일 생성.

전체 개발 프로세스 요약



이 프로세스는 사용자로부터 파일을 받아 텍스트를 추출하고, LLM을 통해 분석하며, 그 결과를 시각화하여 최종적으로 다운로드 가능한 형태로 제공합니다.

확장 및 고도화 전략

RAG (Retrieval-Augmented Generation)



벡터 DB(FAISS) 도입으로
긴 문서 처리.

Security



JWT 기반 사용자 인증 및
RBAC.

Performance



WebSocket 실시간
스트리밍 응답.



감사합니다 (Thank You)

질의응답 (Q&A)

생성형 AI와 모던 웹 스택을 활용한 문서 분석 시스템